This is the **Ultimate Master Guide** for integrating PostgreSQL databases in a Docker environment. It covers every configuration discussed, from basic connectivity to advanced architectural patterns.

Save this entire response as `POSTGRES_INTEGRATION_MASTER_GUIDE.md`.

PostgreSQL Multi-Database Integration Master Guide

This guide provides a complete, end-to-end workflow for connecting PostgreSQL databases hosted in Docker Desktop. We cover **Foreign Data Wrappers (FDW)**, **Cross-Database Relationships**, and **Schema-based Architectures**.

📋 Table of Contents

1. Prerequisites & Environment
2. Phase 1: Docker Infrastructure
3. Phase 2: The Two-Database Integration (FDW)
4. Phase 3: Security-First (Different Teams Workflow)
5. Phase 4: Resolving Foreign Key Conflicts (Soft Keys)
6. Phase 5: The High-Performance Alternative (Schemas)
7. Phase 6: Management via pgAdmin

---

1. Prerequisites & Environment

Create a project folder and a `.env` file to store your credentials.

File: `.env`

```
# Database 1 (Sales / Consumer)
DB1_NAME=sales_db
DB1_USER=sales_admin
DB1_PASS=sales_pass_123
DB1_PORT=5432

# Database 2 (Inventory / Provider)
DB2_NAME=inventory_db
DB2_USER=inventory_admin
DB2_PASS=inventory_pass_456
DB2_PORT=5433

# pgAdmin
PGADMIN_EMAIL=admin@example.com
PGADMIN_PASS=admin_pass
PGADMIN_PORT=8080
```

---

2. Phase 1: Docker Infrastructure

This `docker-compose.yml` sets up two independent database containers and pgAdmin on a shared internal network.

File: `docker-compose.yml`

```yaml
services:
  db1:
    image: postgres:16-alpine
    container_name: sales_container
    environment:
      POSTGRES_DB: ${DB1_NAME}
      POSTGRES_USER: ${DB1_USER}
      POSTGRES_PASSWORD: ${DB1_PASS}
    ports:
      - "${DB1_PORT}:5432"
    volumes:
      - sales_data:/var/lib/postgresql/data
    networks:
      - app_network

  db2:
    image: postgres:16-alpine
    container_name: inventory_container
    environment:
      POSTGRES_DB: ${DB2_NAME}
      POSTGRES_USER: ${DB2_USER}
      POSTGRES_PASSWORD: ${DB2_PASS}
    ports:
      - "${DB2_PORT}:5432"
    volumes:
      - inventory_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB2_USER} -d ${DB2_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app_network

  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin_container
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASS}
    ports:
      - "${PGADMIN_PORT}:80"
    depends_on:
      - db1
      - db2
```

```
    networks:
      - app_network

networks:
  app_network:
    driver: bridge

volumes:
  sales_data:
  inventory_data:
```

---

3. Phase 2: The Two-Database Integration (FDW)

This method allows `db1` to query `db2` directly. Execute these commands in **db1's Query Tool** in pgAdmin.

Step A: Enable Extension & Server

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

-- Use 'db2' as the host because it's the Docker service name
CREATE SERVER inventory_server
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'db2', port '5432', dbname 'inventory_db');
```

Step B: Create User Mapping

```
CREATE USER MAPPING FOR sales_admin
SERVER inventory_server
OPTIONS (user 'inventory_admin', password 'inventory_pass_456');
```

Step C: Selective Importing (Three Options)

Choose how much data you want to bring over:

1. **Import Only Specific Tables (Recommended):**

   ```
   CREATE SCHEMA remote_inventory;
   IMPORT FOREIGN SCHEMA public LIMIT TO (products, stock)
   FROM SERVER inventory_server INTO remote_inventory;
   ```

2. **Import Everything Except Specific Tables:**

```
IMPORT FOREIGN SCHEMA public EXCEPT (internal_logs, sensitive_data)
FROM SERVER inventory_server INTO remote_inventory;
```

3. **Manual Table Definition (Granular Column Control):**

```
CREATE FOREIGN TABLE remote_inventory.product_catalog (
    id int NOT NULL,
    product_name text
) SERVER inventory_server OPTIONS (schema_name 'public', table_name
'products');
```

---

4. Phase 3: Security-First (Different Teams Workflow)

If the databases are managed by different teams, follow this "Least Privilege" model.

1. On Inventory DB (The Provider Team)

Create a Read-Only user and only grant access to specific columns.

```
CREATE USER sales_team_reader WITH PASSWORD 'reader_pass';
GRANT USAGE ON SCHEMA public TO sales_team_reader;
-- Only grant access to the product name and price, hide secret costs!
GRANT SELECT (id, name, price) ON products TO sales_team_reader;
```

2. On Sales DB (The Consumer Team)

Update your User Mapping to use the restricted credentials.

```
CREATE USER MAPPING FOR sales_admin
SERVER inventory_server
OPTIONS (user 'sales_team_reader', password 'reader_pass');
```

---

5. Phase 4: Resolving Foreign Key Conflicts (Soft Keys)

Because **Physical Foreign Keys do not work across databases**, use one of these strategies.

Strategy 1: The Soft Key (Logical Relationship)

- **Database:** Define `product_id` as a standard `INTEGER` (no `REFERENCES`).
- **Application:** Your backend code (Python/Node) queries the Inventory DB to verify the product ID exists before saving the order in the Sales DB.

- **Safety:** Use **Soft Deletes** in the Inventory DB (`deleted_at` column) so the Sales DB never points to a missing ID.

Strategy 2: Trigger-Based Validation

Enforce the rule at the database level using FDW:

```
CREATE OR REPLACE FUNCTION check_inventory_id() RETURNS TRIGGER AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM remote_inventory.products WHERE id =
NEW.product_id) THEN
        RAISE EXCEPTION 'Product ID % does not exist in Inventory!',
NEW.product_id;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_order
BEFORE INSERT ON local_orders
FOR EACH ROW EXECUTE FUNCTION check_inventory_id();
```

---

6. Phase 5: The High-Performance Alternative (Schemas)

If you need **Strict Integrity (Hard Keys)** and **High Speed**, use one database container with multiple schemas.

1. Modified Structure

- **Database:** `app_db`
  - **Schema:** `sales`
  - **Schema:** `inventory`

2. Implementation SQL

```
CREATE SCHEMA inventory;
CREATE SCHEMA sales;

CREATE TABLE inventory.products (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE sales.orders (
    id SERIAL PRIMARY KEY,
    -- NATIVE FOREIGN KEY WORKS HERE!
    product_id INTEGER REFERENCES inventory.products(id) ON DELETE CASCADE,
    customer_name TEXT
);
```

```
-- Querying is seamless
SELECT * FROM sales.orders o JOIN inventory.products p ON o.product_id = p.id;
```

---

7. Phase 6: Management via pgAdmin

1. Open `http://localhost:8080`.
2. **Add Server 1:** Host: `db1`, User: `sales_admin`.
3. **Add Server 2:** Host: `db2`, User: `inventory_admin`.
4. **Verification:**
   - In `db1` -> `Foreign Servers`, you should see `inventory_server`.
   - In `db1` -> `Schemas` -> `remote_inventory` -> `Foreign Tables`, your linked tables should appear.

🚀 Summary: Which Option to Use?

Requirement	Recommendation
Strict Data Integrity (Hard Keys)	**Schemas** (One DB)
Microservices / Different Teams	**FDW** (Two DBs)
High Security / Column Masking	**FDW + Read-Only Mapping**
Simple Prototyping	**Soft Keys**

Addendum

To "redefine" a column as a Soft Key when you are in the **Sales Database** referring to the **Inventory Database**, you simply treat the ID as raw data (a standard `INTEGER`).

Because PostgreSQL cannot "reach out" across the network to verify a Foreign Key in real-time during a standard `INSERT`, you omit the `REFERENCES` clause entirely.

Here is exactly how you do it from the perspective of the **Consumer (Sales) Database**.

---

1. The SQL Definition (The "Soft Key")

When creating your local table in `sales_db`, you define the column type to match the remote ID (usually `INTEGER` or `UUID`), but you leave off the constraint.

```
-- Run this in SALES_DB
CREATE TABLE public.orders (
    order_id SERIAL PRIMARY KEY,

    -- REDEFINED: No 'REFERENCES inventory_db...' here!
    -- We call this a "Soft Key" or "Logical Key"
```

```
    product_id INTEGER NOT NULL,

    customer_name TEXT,
    quantity INTEGER
);
```

---

2. How you "Connect" them (The Join)

Even though there is no formal link in the table definition, you create the link **during the query** using the Foreign Data Wrapper (FDW) schema we set up earlier.

```
-- Querying from SALES_DB
SELECT
    o.order_id,
    o.customer_name,
    p.name AS product_name,  -- This comes from the OTHER database
    p.price                 -- This comes from the OTHER database
FROM public.orders o
JOIN remote_inventory.products p ON o.product_id = p.id;
```

---

3. How to "Simulate" the Reference (Integrity)

Since you don't have a `REFERENCES` constraint to stop users from entering a fake `product_id`, you have two ways to handle it from your side (the Sales DB):

Option A: The "Check" Trigger (Internal Validation)

If you want the Sales Database to error out if someone tries to insert a `product_id` that doesn't exist in the Inventory Database:

```
CREATE OR REPLACE FUNCTION validate_remote_product()
RETURNS TRIGGER AS $$
BEGIN
    -- Look into the FDW table to see if the ID exists
    IF NOT EXISTS (SELECT 1 FROM remote_inventory.products WHERE id =
NEW.product_id) THEN
        RAISE EXCEPTION 'Product ID % does not exist in the Inventory Database',
NEW.product_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_orders_product_check
BEFORE INSERT OR UPDATE ON public.orders
FOR EACH ROW EXECUTE FUNCTION validate_remote_product();
```

Option B: The Application Validation (External Validation)

If you are using a backend (Node, Python, PHP), your code handles the check:

1. **App:** `SELECT id FROM remote_inventory.products WHERE id = 50;`
2. **App:** If result is null, stop and show error.
3. **App:** If result exists, `INSERT INTO public.orders (product_id, ...) VALUES (50, ...);`

---

4. Pro-Tip: Naming Conventions

To make it clear to other developers that this `product_id` is a "Foreign" key that isn't enforced locally, many teams use a specific naming convention:

- Instead of `product_id`, name it `remote_product_id` or `ext_product_id`.
- Add a **comment** to the column in the database:

```
COMMENT ON COLUMN public.orders.product_id IS 'Soft Key: References
products(id) in inventory_db via FDW';
```

Summary of the "Redefinition"

1. **Type:** Must match the remote side (usually `INT`).
2. **Constraint:** Remove `REFERENCES ...`.
3. **Bridge:** Use the `JOIN` on the FDW table for reports.
4. **Enforcement:** Use a **Trigger** if you want the DB to handle it, or **App Logic** if you want your code to handle it.
5. **Naming Conventions:** To make it clear to other developers that this `product_id` is a "Foreign" key that isn't enforced locally, many teams use a specific naming convention:
   - Instead of `product_id`, name it `remote_product_id` or `ext_product_id`.
   - Add a **comment** to the column in the database:

     ```
     COMMENT ON COLUMN public.orders.product_id IS 'Soft Key: References
     products(id) in inventory_db via FDW';
     ```

   - Use a **View** to hide the complexity:

     ```
     CREATE VIEW public.orders_with_products AS
     SELECT
         o.order_id,
         o.customer_name,
         p.name AS product_name,
         p.price
     FROM public.orders o
     JOIN remote_inventory.products p ON o.product_id = p.id;
     ```

Integration from 'backupxxxx' file

To integrate a backup file into your Dockerized PostgreSQL setup, you need to follow a three-step process: **Upload**, **Restore**, and **Link**.

Since you have a backup file (let's call it `backup.sql` or `backup.dump`), we will restore it into **Database 2 (`db2`)** and then link it to **Database 1 (`db1`)**.

Step 1: Move the Backup into Docker

Docker containers cannot "see" files on your Windows/Mac desktop unless you move them or mount them.

Option A: The Command Line (Fastest) Open your terminal/command prompt in the folder where your backup file is located and run:

```
docker cp backupxxxx sales_container:/tmp/backupxxxx
```

(Replace `sales_container` with the `container_name` from your docker-compose for db2).

Option B: The Volume Folder If your `docker-compose` has a volume like `./db2_init:/docker-entrypoint-initdb.d`, just move the file into that folder on your computer. If it is a `.sql` file, Postgres will try to run it automatically when the container starts for the first time.

Step 2: Restore the Backup via pgAdmin

Once the file is "inside" the environment, use pgAdmin to restore it.

1. Open **pgAdmin**.
2. Right-click on your **Target Database** (e.g., `inventory_db` on `db2`).
3. Select **Restore**.
4. In the **Filename** field, click the folder icon.
    - If you used `docker cp`, look in `/tmp/`.
    - If you mapped a volume, look in the mapped path.
5. Select your file and click **Restore**.

> **Note:** If the backup was created using the "Plain" format in pgAdmin, use the **Query Tool** to open the file and hit **Play**. If it was "Custom" or "Tar" format, use the **Restore** menu.

Step 3: Create the Integration (The "Link")

Now that your data is living inside `db2`, you need to tell `db1` how to read it using the **Foreign Data Wrapper** method we discussed.

Run this in the Query Tool of `db1`:

```sql
-- 1. Enable the bridge tool
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

-- 2. Define the server (use the docker service name 'db2')
CREATE SERVER restored_server
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'db2', port '5432', dbname 'inventory_db');

-- 3. Map your credentials
CREATE USER MAPPING FOR sales_admin
SERVER restored_server
OPTIONS (user 'inventory_admin', password 'inventory_pass_456');

-- 4. Import the specific tables from the backup
-- IMPORTANT: Look at the restored database in pgAdmin.
-- If the backup created a schema named 'public', use 'public'.
-- If it created a schema named 'client_data', use that.
CREATE SCHEMA restored_data;

IMPORT FOREIGN SCHEMA public
LIMIT TO (table_name_from_backup)
FROM SERVER restored_server
INTO restored_data;
```

Step 4: Resolve the Foreign Key (Soft Key)

Now that the backup data is visible in `db1`, you can link your local tables to it.

If your backup contains a table of `products` and you want your local `orders` table to point to them:

1. **Check the ID type:** Look at the `id` column in `restored_data.products`. Is it an Integer? A UUID?
2. **Create your local table:**

```sql
CREATE TABLE local_orders (
    id SERIAL PRIMARY KEY,
    -- Redefine the product_id to match the backup's ID type
    backup_product_id INTEGER,
    customer_name TEXT
);
```

3. **Perform the Integration Join:**

```sql
SELECT
    o.customer_name,
    p.name as product_from_backup
FROM local_orders o
JOIN restored_data.products p ON o.backup_product_id = p.id;
```

Troubleshooting Common Backup Integration Issues:

- **Role Not Found:** If the backup was made from a database where the user was `boss_man`, and your Docker user is `admin`, the restore might show errors.
  - *Fix:* In the pgAdmin Restore dialog, go to **Restore options** and check **Do not save owner**.
- **Schema Confusion:** If your backup contains multiple schemas, you must run the `IMPORT FOREIGN SCHEMA` command for **each** schema you want to link.
- **Search Path:** If you want to query the restored tables easily without typing `restored_data.`, run:

  ```
  SET search_path TO public, restored_data;
  ```

Summary of the Workflow:

1. **Restore** the backup into the second container (`db2`).
2. **Link** the first container (`db1`) to the second using `postgres_fdw`.
3. **Map** the foreign tables into a local schema.
4. **Join** them using "Soft Keys" (Standard Integers) in your queries.

End of Guide.